

Étude de la confiance accordée à un humain vs une IA

[À MODIFIER: Prénom NOM], [À MODIFIER: Prénom NOM]

Master 1 Génie Logiciel
[À MODIFIER: Université]

24 mai 2026

- 1 Introduction
- 2 Contexte du Projet
- 3 Développement
 - Architecture générale
 - Plateforme Web
 - Application Python
 - Intégration Web ↔ Python
- 4 Protocole Expérimental
- 5 Résultats Attendus
- 6 Conclusion

- **Question fondamentale** : Les utilisateurs accordent-ils différemment leur confiance à un adversaire humain vs une IA ?
- **Contexte** : Omniprésence de l'IA dans les interactions quotidiennes
- **Enjeu** : Conception d'interfaces homme-machine éthiques et fiables

Théorie de l'attribution

- Heider (1958), Weiner (1985)
- Explications causales biaisées pour les machines

Étude Ullman et al. (2014)

- Yale University
- Confiance vs tromperie du robot

Anthropomorphisme inversé

- Standards moraux différents selon la cible
- Machines jugées plus sévèrement

- H₁** Les participants expriment une évaluation de déloyauté **significativement plus élevée** pour le CheatBot que pour le même comportement attribué à un humain
- H_{2a}** L'écart de confiance **diminue** au fil des tours (apprentissage)
- H_{2b}** L'ordre de présentation **influence** les perceptions ultérieures

Phase 1 : Robot Pepper

- Objectif initial : humain vs humanoïde vs digital
- Intégration avec robot social SoftBank

Problèmes rencontrés

- Parties trop longues (15-30s latence)
- Confonds expérimentaux
- Expérience frustrante

Phase 2 : Pivot vers digital

- Maintien de Python (infrastructure Pepper déjà développée)
- Ajout de couche web (Flask)
- Avantage : contrôle rigoureux, reproductibilité, scalabilité

Image / Schéma : Architecture Pepper Battleship (Backend Python + Frontend Web)

Backend Python

- Python 2.7+
- Architecture modulaire
- SQLite
- Socket TCP/IP

Frontend Web

- Flask 2.3+
- API REST
- HTML5 + CSS3
- JavaScript vanilla

Communication

- API REST JSON
- Session management
- Base SQLite centralisée

Image / Schéma : Schéma : Intégration Flask - Backend Python - SQLite

Sections principales :

- ➊ Navigation
- ➋ Héros (appel à l'action)
- ➌ 6 cartes de fonctionnalités
- ➍ Tutoriel (4 étapes)
- ➎ Section recherche

*Image / Schéma :
Capture d'écran
Landing Page*

Machine à états (5 phases) :

- ➊ **Setup** : Entrée du nom du joueur
- ➋ **Persona** : Sélection du contexte (humain vs robot)
- ➌ **Grid** : Validation configuration de flotte
- ➍ **Playing** : Jeu et collecte de scores de confiance
- ➎ **GameOver** : Résultats et débriefing

Image / Schéma : Schéma : État transitions de la machine à états

Grilles de jeu (10×10)

Grille du joueur :

- Ma flotte (6 bateaux)
- Tirs ennemis reçus
- Symboles : X C *

Grille de suivi :

- Mes tirs effectués
- Résultats (touché/raté/coulé)
- Visualisation en temps réel

Image / Schéma : Capture d'écran : Deux grilles côte à côte

Après chaque tour du bot :

“Dans quelle mesure pensez-vous que le bot a triché ?”

Score	Interprétation
0	Pas du tout triché
1-2	Peu probable
3	Incertain
4-5	Très suspect / Certainement triché

Image / Schéma : Capture d'écran : Widget d'évaluation

Routes principales :

- GET / : Landing page
- POST /api/game/new : Créer partie
- POST /api/game/{id}/shoot : Tir joueur
- POST /api/game/{id}/bot-turn : Tour du bot
- POST /api/game/{id}/turn-trust : Enregistrer score
- GET /api/stats : Statistiques globales

Image / Schéma : Schéma : Flux d'appels API pour une partie

Image / Schéma : Schéma : Arborescence des modules

GameMaster (Orchestrateur principal)

Image / Schéma : Schéma : Rôles et responsabilités du GameMaster

Le cœur du protocole expérimental

Processus par tour :

- ➊ Accès à la grille ennemie (si mode digital)
- ➋ Identification des cases de bateaux non touchées
- ➌ Sélection de N cases bateaux (succès)
- ➍ Sélection de $4 - N$ cases vides (échecs)
- ➎ Mélange aléatoire (éviter patterns)
- ➏ Exécution des 4 tirs

Image / Schéma : Schéma : Algorithme execute_turn du CheatBot

Garantie de contrôle expérimental :

- Quota prédéfini par tour (ex : [2, 1, 3, 2, 1, ...])
- **Même séquence** utilisée dans les deux conditions
- Adaptation automatique si moins de bateaux disponibles
- Communication socket optionnelle vers Pepper

Résultat : Comportement observable **strictement identique** dans les deux parties, seul le label varie.

*Image / Schéma : Schéma : Comparaison quotas Partie 1 vs
Partie 2*

Image / Schéma : Diagramme UML : Hiérarchie Player

Grid (10×10)

- Navires placés
- État de chaque case
- Méthode `shoot(x,y)`

Ship

- Position, orientation
- Taille (5, 4, 3, 3, 2)
- Détection coulage

Position

- Coordonnées (x, y)
- Validation limites

Response

- Message (touché/coulé/raté)
- État après tir

Image / Schéma : Schéma : Relations entre classes

Schéma relationnel :

Table Game

- game_id (PK)
- player_name
- player_type
- date_played
- winner

Table Turn

- turn_id (PK)
- game_id (FK)
- turn_number
- bot_quota
- trust_score

Table BotShot

- shot_id (PK)
- turn_id (FK)
- shot_number
- pos_x, pos_y
- result

Image / Schéma : Schéma : Diagramme E-R

Mode Console

- Affichage textuel
- Interaction CLI
- Tests et développement

Mode Graphique (Tkinter)

- Canvases 10×10
- Visualisation intuitive
- Intégration questionnaire

Image / Schéma : Capture d'écran : Interface Tkinter

Flux de données complet :

- 1 Utilisateur accède à /game (Flask)
- 2 Flask initialise GameMaster en Python
- 3 Tirs joueur → API /shoot → logique Python
- 4 Tour du bot → appel CheatBot.get_case_played()
- 5 Résultats → enregistrement SQLite
- 6 Scores de confiance → base de données

Image / Schéma : Schéma : Flux complet client-serveur

Avantages de cette architecture

- **Interface moderne** : Web pour UX optimale et recrutement en ligne
- **Logique robuste** : Python éprouvé et testé
- **Contrôle rigoureux** : Comportement identique dans les deux conditions
- **Reproductibilité** : Code centralisé et versionné
- **Scalabilité** : Collecte de données auprès de nombreux participants
- **Flexibilité future** : Intégration possible avec Pepper (socket TCP/IP)

Design quasi-expérimental

Plan intra-groupe (within-subjects) :

Chaque participant joue **2 parties** avec comportement adversaire **identique**

	Partie 1	Partie 2
Condition A	Adversaire X (Label 1)	Adversaire Y (Label 2)
Condition B	Adversaire Y (Label 2)	Adversaire X (Label 1)

Label 1 = [À MODIFIER: Humain ou CheatBot]

Label 2 = [À MODIFIER: CheatBot ou Humain]

Image / Schéma : Schéma : Contrebalancement des conditions

Déroulement des parties

Phases d'une partie :

- 1 Sélection du nom du joueur
- 2 **[À MODIFIER: [Questionnaire démographique - À préciser]]**
- 3 Sélection de la configuration de flotte (10 grilles prédéfinies)
- 4 Combat (alternance joueur $\leftrightarrow$ bot, 4 tirs/tour)
- 5 Évaluation de confiance après chaque tour du bot (0-5)
- 6 Fin : Victoire ou défaite
- 7 **[À MODIFIER: [Questionnaire post-expérience - À développer]]**

Image / Schéma : Timeline : Déroulement temporel d'une partie

Variable dépendante principale :

À la fin de chaque tour du bot, le joueur répond :

“Dans quelle mesure pensez-vous que votre adversaire a triché ?”

Échelle de Likert 0-5 :

- 0 = Pas du tout triché
- 1-2 = Peu probable
- 3 = Incertain
- 4-5 = Très suspect / Certainement triché

Enregistrement : Horodaté et stocké immédiatement en base

Garantie d'équivalence comportementale :

- **Même adversaire** : Code CheatBot identique
- **Même quota** : Séquence $[q1, q2, \dots, qN]$ répétée
- **Même grilles** : Configurations prédéfinies identiques
- **Même ordre** : Séquence de quotas appliquée dans l'ordre

Variable indépendante manipulée :

- Label de l'adversaire (Humain vs CheatBot)

Image / Schéma : Schéma : Comparaison des deux conditions

Analyses principales :

- ① Test de comparaison des moyennes (scores de confiance par condition)
- ② Calcul de la magnitude de l'effet (Cohen's d)
- ③ Analyse temporelle (évolution des scores au fil des tours)
- ④ **[À MODIFIER: [Analyses statistiques supplémentaires à définir]]**

Hypothèses à tester :

- H_1 : Différence significative entre les deux labels
- H_{2a} : Diminution de l'écart au fil du temps
- H_{2b} : Effet d'ordre de présentation

*Image / Schéma : Graphique attendu : Scores de confiance
par condition*

Si H_1 est confirmée :

- Les utilisateurs appliquent des standards différents selon le label
- Biais systématique dans la perception de tromperie artificielle
- Implications pour le design éthique d'IA

Si H_1 est réfutée :

- La perception de déloyauté est basée sur le comportement, pas le label
- Questions sur la nature du biais anthropomorphe

Résultats secondaires :

- Profils individuels de sensibilité à la triche
- Rôle de l'expérience préalable avec l'IA
- **[À MODIFIER: [Analyses exploratoires à définir]]**

Apports scientifiques :

- **Théorique** : Éclairage sur les mécanismes de confiance asymétrique
- **Méthodologique** : Protocole reproductible et rigoureux
- **Applicatif** : Recommandations pour design d'IA responsable

Particularités du projet :

- Architecture web+Python flexible et scalable
- Contrôle expérimental absolu du comportement adversaire
- Collecte de données centralisée
- Intégrabilité future avec robot physique

- **Extension** : Réintégration du robot Pepper (bottleneck résolu)
- **Variations** : Différents contextes compétitifs (poker, jeux de données)
- **Longitudinal** : Suivi de la confiance chez mêmes participants
- **Démographique** : Étude des facteurs individuels modulant le biais

Pepper Battleship

Détection de triche et confiance homme-machine

[À MODIFIER: Université - Master 1 Génie Logiciel]

Questions ?